l-for-distributed-database-systems

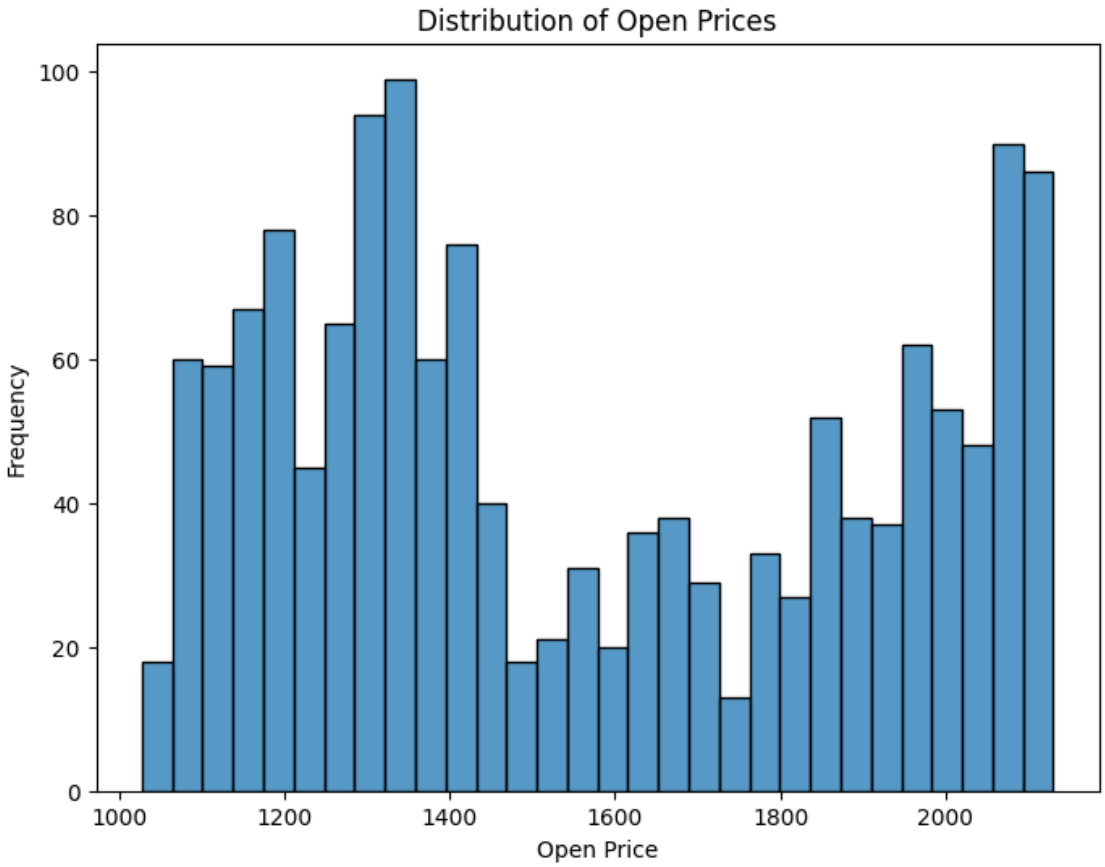
October 18, 2024

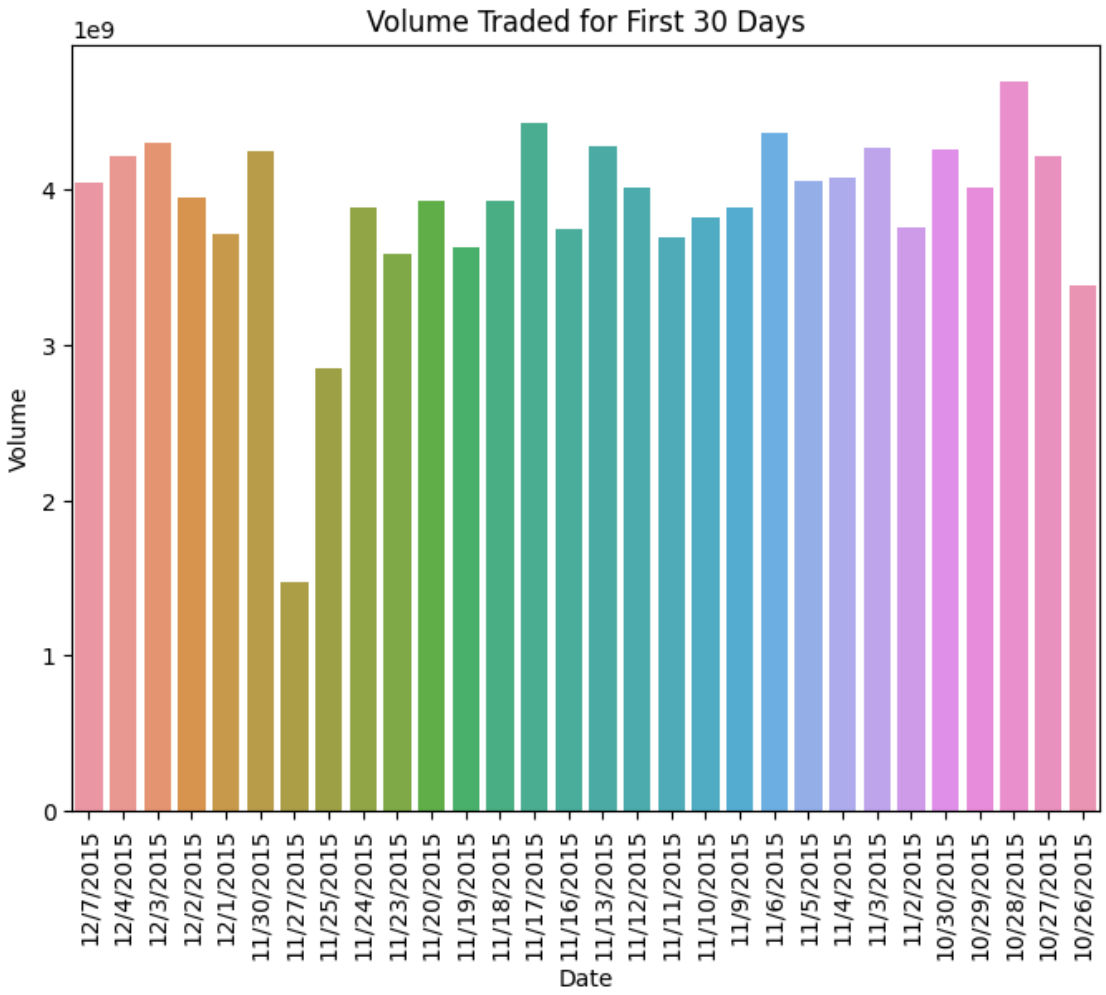
[2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from scipy.optimize import linprog

# Load the dataset
data = pd.read_csv('/kaggle/input/yahoo-data/yahoo.csv')

# Inspect the first few rows
print("First few rows of the dataset:")
print(data.head())

# Check for missing values and data types
print("\nData Info:")
print(data.info())

# Describe the data to get a basic statistical summary
print("\nData Description:")
print(data.describe())

# Handle missing values (if any)
data.fillna(method='ffill', inplace=True)  # Forward fill as an example

# Data Visualization
# Histogram to visualize the distribution of the 'Open' column
plt.figure(figsize=(8,6))
sns.histplot(data['Open'], bins=30)
plt.title('Distribution of Open Prices')
plt.xlabel('Open Price')
plt.ylabel('Frequency')
plt.show()
```

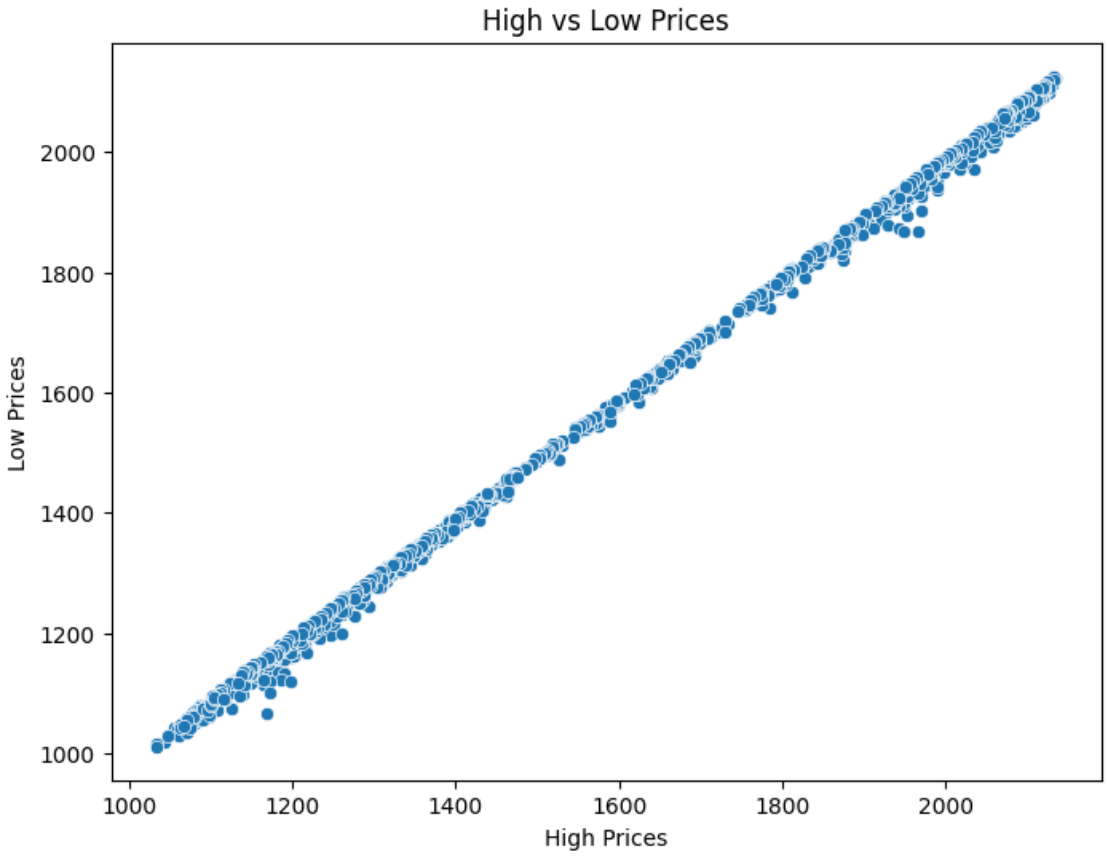
```
# Bar plot for categorical-like data (using the 'Volume' column as an example)
plt.figure(figsize=(8,6))
sns.barplot(x=data['Date'][:30], y=data['Volume'][:30])  # Only the first 30␣
 ↪dates to avoid overcrowding
plt.title('Volume Traded for First 30 Days')
plt.xlabel('Date')
plt.ylabel('Volume')
plt.xticks(rotation=90)  # Rotate the dates for better readability
plt.show()

# Scatter plot to examine the correlation between 'High' and 'Low' prices
plt.figure(figsize=(8,6))
sns.scatterplot(x='High', y='Low', data=data)
plt.title('High vs Low Prices')
plt.xlabel('High Prices')
plt.ylabel('Low Prices')
plt.show()
```

```
First few rows of the dataset:
        Date         Open         High          Low        Close    Adj Close  \
0  12/7/2015  2090.419922  2090.419922  2066.780029  2077.070068  2077.070068
1  12/4/2015  2051.239990  2093.840088  2051.239990  2091.689941  2091.689941
2  12/3/2015  2080.709961  2085.000000  2042.349976  2049.620117  2049.620117
3  12/2/2015  2101.709961  2104.270020  2077.110107  2079.510010  2079.510010
4  12/1/2015  2082.929932  2103.370117  2082.929932  2102.629883  2102.629883

       Volume
0  4043820000
1  4214910000
2  4306490000
3  3950640000
4  3712120000

Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1493 entries, 0 to 1492
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Date       1493 non-null   object
 1   Open       1493 non-null   float64
 2   High       1493 non-null   float64
 3   Low        1493 non-null   float64
 4   Close      1493 non-null   float64
 5   Adj Close  1493 non-null   float64
 6   Volume     1493 non-null   int64
dtypes: float64(5), int64(1), object(1)
```

```
memory usage: 81.8+ KB
None

Data Description:
              Open         High          Low        Close    Adj Close  \
count  1493.000000  1493.000000  1493.000000  1493.000000  1493.000000
mean   1564.805712  1573.101540  1556.045962  1565.370924  1565.370924
std     344.474195   344.719565   344.233831   344.453792   344.453792
min    1027.650024  1032.949951  1010.909973  1022.580017  1022.580017
25%    1277.030029  1283.930054  1267.400024  1277.300049  1277.300049
50%    1459.369995  1462.430054  1452.060059  1459.369995  1459.369995
75%    1911.770020  1927.209961  1902.010010  1913.849976  1913.849976
max    2130.360107  2134.719971  2126.060059  2130.820068  2130.820068

             Volume
count  1.493000e+03
mean   3.778865e+09
std    8.865364e+08
min    5.362000e+08
25%    3.252780e+09
50%    3.673450e+09
75%    4.214910e+09
max    1.061781e+10
/tmp/ipykernel_30/3677362947.py:27: FutureWarning: DataFrame.fillna with
'method' is deprecated and will raise in a future version. Use obj.ffill() or
obj.bfill() instead.
  data.fillna(method='ffill', inplace=True)  # Forward fill as an example
/opt/conda/lib/python3.10/site-packages/seaborn/_oldcore.py:1119: FutureWarning:
use_inf_as_na option is deprecated and will be removed in a future version.
Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```


Distribution of Open Prices
Frequency
Open Price
0
20
40
60
80
100
1000
1200
1400
1600
1800
2000

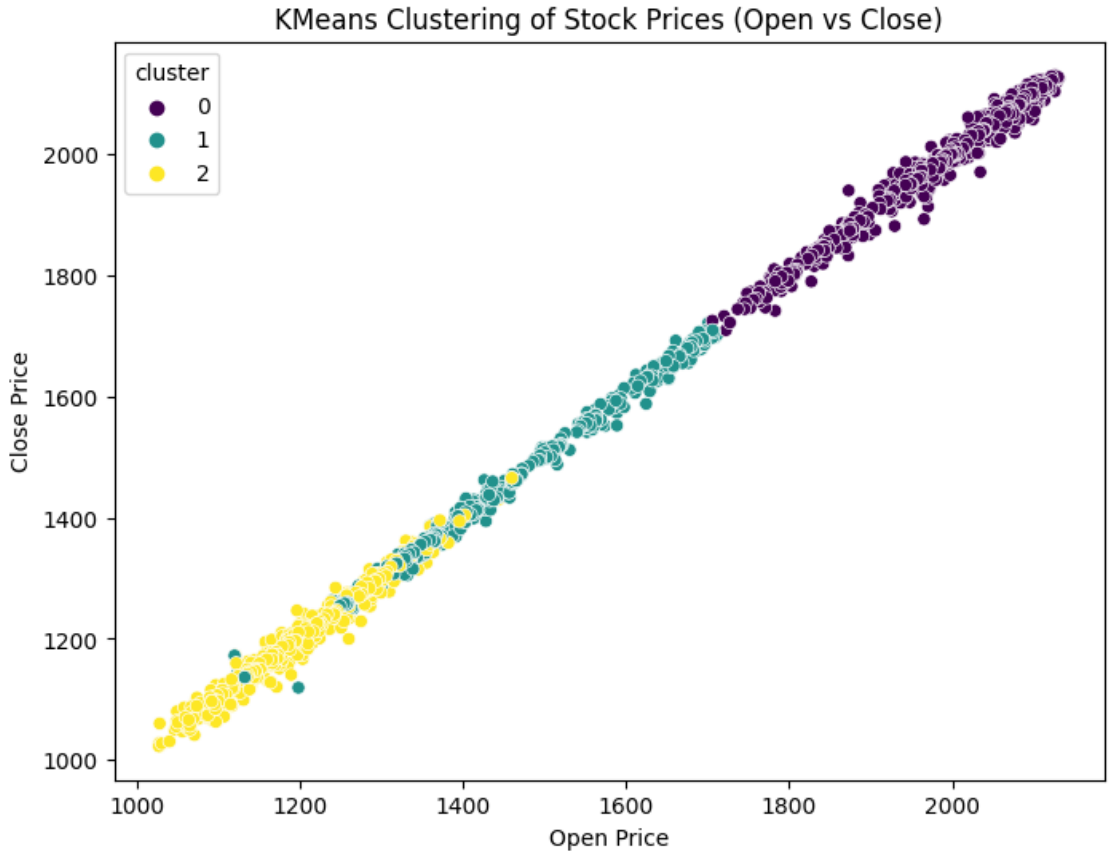

Volume Traded for First 30 Days
1e9
Volume
0
1
2
3
4
Date
12/7/2015
12/4/2015
12/3/2015
12/2/2015
12/1/2015
11/30/2015
11/27/2015
11/25/2015
11/24/2015
11/23/2015
11/20/2015
11/19/2015
11/18/2015
11/17/2015
11/16/2015
11/13/2015
11/12/2015
11/11/2015
11/10/2015
11/9/2015
11/6/2015
11/5/2015
11/4/2015
11/3/2015
11/2/2015
10/30/2015
10/29/2015
10/28/2015
10/27/2015
10/26/2015



[5]:
```
# Clustering Model using KMeans
# Scaling the data for better clustering results
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data[['Open', 'High', 'Low', 'Close',␣
 ↪'Volume']])

# Applying KMeans clustering
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(scaled_data)

# Add the cluster labels to the original dataset
data['cluster'] = clusters

# Visualize the clusters using 'Open' and 'Close' prices
plt.figure(figsize=(8,6))
sns.scatterplot(x='Open', y='Close', hue='cluster', data=data,␣
 ↪palette='viridis')
plt.title('KMeans Clustering of Stock Prices (Open vs Close)')
plt.xlabel('Open Price')
plt.ylabel('Close Price')
```

```
plt.show()
```

```
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
  warnings.warn(
```



[7]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Load the dataset
data = pd.read_csv('/kaggle/input/yahoo-data/yahoo.csv')

# Create a target column: 1 if the Close price increased compared to the␣
 ↪previous day, 0 otherwise
```

```
data['price_change'] = np.where(data['Close'].diff() > 0, 1, 0)

# Feature Engineering:
# 1. Percentage change between Open and Close prices
data['pct_change'] = (data['Close'] - data['Open']) / data['Open'] * 100

# 2. 5-day rolling average of the Close price
data['rolling_avg_5'] = data['Close'].rolling(window=5).mean()

# 3. Daily volatility (High - Low)
data['volatility'] = data['High'] - data['Low']

# Drop the first few rows with NaN values due to rolling window
data.dropna(inplace=True)

# Define features (X) and target (y)
X = data[['Open', 'High', 'Low', 'Close', 'Volume', 'pct_change',␣
 ↪'rolling_avg_5', 'volatility']]
y = data['price_change']  # Use the target column

# Splitting the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=42)

# Train a RandomForest Classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# Make predictions and evaluate
y_pred = clf.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.56      0.75      0.64       233
           1       0.58      0.37      0.45       214

    accuracy                           0.57       447
   macro avg       0.57      0.56      0.55       447
weighted avg       0.57      0.57      0.55       447
```

[8]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
```

```python
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split

# Oversample the minority class using SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Split the resampled data
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,␣
 ↪test_size=0.3, random_state=42)

# Hyperparameter tuning using GridSearchCV
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10]
}

# Create a GridSearchCV object
grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param_grid,␣
 ↪cv=5, n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)

# Best model
best_rf = grid_search.best_estimator_

# Predictions
y_pred = best_rf.predict(X_test)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits

Classification Report:
              precision    recall  f1-score   support

           0       0.58      0.63      0.61       249
           1       0.58      0.52      0.55       240

    accuracy                           0.58       489
   macro avg       0.58      0.58      0.58       489
weighted avg       0.58      0.58      0.58       489
```

[9]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE

# Load the dataset
data = pd.read_csv('/kaggle/input/yahoo-data/yahoo.csv')

# Create a target column: 1 if the Close price increased compared to the␣
 ↪previous day, 0 otherwise
data['price_change'] = np.where(data['Close'].diff() > 0, 1, 0)

# Feature Engineering:
# 1. Percentage change between Open and Close prices
data['pct_change'] = (data['Close'] - data['Open']) / data['Open'] * 100

# 2. 5-day rolling average of the Close price
data['rolling_avg_5'] = data['Close'].rolling(window=5).mean()

# 3. Daily volatility (High - Low)
data['volatility'] = data['High'] - data['Low']

# Drop the first few rows with NaN values due to rolling window
data.dropna(inplace=True)

# Define features (X) and target (y)
X = data[['Open', 'High', 'Low', 'Close', 'Volume', 'pct_change',␣
 ↪'rolling_avg_5', 'volatility']]
y = data['price_change']

# Oversample the minority class using SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Split the resampled data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,␣
 ↪test_size=0.3, random_state=42)

# Train an XGBoost Classifier
xgb = XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=6,␣
 ↪random_state=42, use_label_encoder=False)
xgb.fit(X_train, y_train)

# Make predictions and evaluate
y_pred = xgb.predict(X_test)
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.60      0.61      0.60       249
           1       0.59      0.57      0.58       240

    accuracy                           0.59       489
   macro avg       0.59      0.59      0.59       489
weighted avg       0.59      0.59      0.59       489
```

[10]:
```
from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBClassifier

# Define the parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5, 7, 10],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0]
}

# Instantiate XGBoost classifier
xgb = XGBClassifier(random_state=42, use_label_encoder=False)

# Randomized search for hyperparameter tuning
random_search = RandomizedSearchCV(xgb, param_distributions=param_grid,␣
 ↪n_iter=20, scoring='accuracy', n_jobs=-1, cv=5, random_state=42)
random_search.fit(X_train, y_train)

# Best parameters found by the search
best_xgb = random_search.best_estimator_

# Make predictions and evaluate
y_pred = best_xgb.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support
```

```
              0       0.61      0.62      0.61       249
              1       0.60      0.58      0.59       240

       accuracy                           0.60       489
      macro avg       0.60      0.60      0.60       489
   weighted avg       0.60      0.60      0.60       489
```

[11]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import RandomizedSearchCV

# Load the dataset
data = pd.read_csv('/kaggle/input/yahoo-data/yahoo.csv')

# Create a target column: 1 if the Close price increased compared to the␣
 ↪previous day, 0 otherwise
data['price_change'] = np.where(data['Close'].diff() > 0, 1, 0)

# Feature Engineering:
# 1. Percentage change between Open and Close prices
data['pct_change'] = (data['Close'] - data['Open']) / data['Open'] * 100

# 2. 5-day rolling average of the Close price
data['rolling_avg_5'] = data['Close'].rolling(window=5).mean()

# 3. Daily volatility (High - Low)
data['volatility'] = data['High'] - data['Low']

# Technical Indicators:
# 4. RSI (Relative Strength Index) calculation
window_length = 14
close_delta = data['Close'].diff()
gain = (close_delta.where(close_delta > 0, 0)).rolling(window=window_length).
 ↪mean()
loss = (-close_delta.where(close_delta < 0, 0)).rolling(window=window_length).
 ↪mean()
rs = gain / loss
data['RSI'] = 100 - (100 / (1 + rs))

# 5. MACD (Moving Average Convergence Divergence)
ema_12 = data['Close'].ewm(span=12, adjust=False).mean()
ema_26 = data['Close'].ewm(span=26, adjust=False).mean()
```

```
data['MACD'] = ema_12 - ema_26

# 6. Bollinger Bands
data['20_day_ma'] = data['Close'].rolling(window=20).mean()
data['20_day_std'] = data['Close'].rolling(window=20).std()
data['upper_band'] = data['20_day_ma'] + (data['20_day_std'] * 2)
data['lower_band'] = data['20_day_ma'] - (data['20_day_std'] * 2)

# Drop rows with NaN values generated by rolling calculations
data.dropna(inplace=True)

# Define features (X) and target (y)
X = data[['Open', 'High', 'Low', 'Close', 'Volume', 'pct_change',␣
 ↪'rolling_avg_5', 'volatility', 'RSI', 'MACD', 'upper_band', 'lower_band']]
y = data['price_change']

# Oversample the minority class using SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Split the resampled data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,␣
 ↪test_size=0.3, random_state=42)

# Hyperparameter tuning with RandomizedSearchCV for XGBoost
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5, 7, 10],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0]
}

xgb = XGBClassifier(random_state=42, use_label_encoder=False)
random_search = RandomizedSearchCV(xgb, param_distributions=param_grid,␣
 ↪n_iter=20, scoring='accuracy', n_jobs=-1, cv=5, random_state=42)
random_search.fit(X_train, y_train)

# Best model
best_xgb = random_search.best_estimator_

# Make predictions and evaluate
y_pred = best_xgb.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.64      0.56      0.59       245
           1       0.60      0.68      0.63       240

    accuracy                           0.61       485
   macro avg       0.62      0.62      0.61       485
weighted avg       0.62      0.61      0.61       485
```

[12]:
```python
from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split

# Load the dataset and continue from the feature engineering and target creation
# (Assuming data preparation steps are same as before)

# Oversample the minority class using SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Split the resampled data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,␣
 ↪test_size=0.3, random_state=42)

# Define the parameter grid for GridSearchCV
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0]
}

# Initialize the XGBoost classifier
xgb = XGBClassifier(random_state=42, use_label_encoder=False)

# Initialize GridSearchCV with 5-fold cross-validation
grid_search = GridSearchCV(estimator=xgb, param_grid=param_grid, cv=5,␣
 ↪n_jobs=-1, verbose=1, scoring='accuracy')

# Fit the grid search to the training data
grid_search.fit(X_train, y_train)
```

```
# Retrieve the best parameters
print(f"Best Parameters: {grid_search.best_params_}")

# Best model from grid search
best_xgb = grid_search.best_estimator_

# Make predictions using the best model
y_pred = best_xgb.predict(X_test)

# Print classification report to evaluate the model's performance
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Fitting 5 folds for each of 243 candidates, totalling 1215 fits
Best Parameters: {'colsample_bytree': 1.0, 'learning_rate': 0.01, 'max_depth':
3, 'n_estimators': 300, 'subsample': 0.6}

Classification Report:
              precision    recall  f1-score   support

           0       0.64      0.56      0.60       245
           1       0.60      0.67      0.63       240

    accuracy                           0.62       485
   macro avg       0.62      0.62      0.62       485
weighted avg       0.62      0.62      0.62       485
```

[13]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE

# SMOTE for class balancing
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,␣
 ↪test_size=0.3, random_state=42)

# Initialize Logistic Regression model
log_reg = LogisticRegression(max_iter=1000, random_state=42)

# Train Logistic Regression
log_reg.fit(X_train, y_train)
```

```python
# Make predictions and evaluate
y_pred = log_reg.predict(X_test)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.51      1.00      0.67       245
           1       0.00      0.00      0.00       240

    accuracy                           0.51       485
   macro avg       0.25      0.50      0.34       485
weighted avg       0.26      0.51      0.34       485
```

```
/opt/conda/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344:
UndefinedMetricWarning: Precision and F-score are ill-defined and being set to
0.0 in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/opt/conda/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344:
UndefinedMetricWarning: Precision and F-score are ill-defined and being set to
0.0 in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/opt/conda/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344:
UndefinedMetricWarning: Precision and F-score are ill-defined and being set to
0.0 in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

[14]:
```python
import lightgbm as lgb
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE

# SMOTE for class balancing
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,
 ↪test_size=0.3, random_state=42)
```

```python
# Initialize LightGBM model
lgbm = lgb.LGBMClassifier(n_estimators=300, learning_rate=0.01, max_depth=3, subsample=0.6, colsample_bytree=1.0, random_state=42)

# Train LightGBM
lgbm.fit(X_train, y_train)

# Make predictions and evaluate
y_pred = lgbm.predict(X_test)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
[LightGBM] [Info] Number of positive: 568, number of negative: 563
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of
testing was 0.001697 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3060
[LightGBM] [Info] Number of data points in the train set: 1131, number of used
features: 12
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.502210 -> initscore=0.008842
[LightGBM] [Info] Start training from score 0.008842
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

Classification Report:
              precision    recall  f1-score   support

           0       0.63      0.58      0.60       245
           1       0.60      0.65      0.62       240

    accuracy                           0.61       485
   macro avg       0.61      0.61      0.61       485
weighted avg       0.61      0.61      0.61       485
```

[15]:
```python
from sklearn.svm import SVC
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE

# SMOTE for class balancing
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,␣
 ↪test_size=0.3, random_state=42)

# Initialize SVM model with RBF kernel
svm = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)

# Train SVM
svm.fit(X_train, y_train)

# Make predictions and evaluate
```

```
y_pred = svm.predict(X_test)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.51      0.86      0.64       245
           1       0.54      0.17      0.26       240

    accuracy                           0.52       485
   macro avg       0.53      0.51      0.45       485
weighted avg       0.53      0.52      0.45       485
```

[16]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
import lightgbm as lgb
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from imblearn.over_sampling import SMOTE

# Define models to compare
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'XGBoost': XGBClassifier(n_estimators=100, random_state=42,␣
 ↪use_label_encoder=False),
    'LightGBM': lgb.LGBMClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(kernel='rbf', random_state=42)
}

# Function to train and evaluate models
def evaluate_models(models, X_train, X_test, y_train, y_test):
    results = []
    for name, model in models.items():
        # Train the model
        model.fit(X_train, y_train)

        # Make predictions
        y_pred = model.predict(X_test)
```

```python
        # Evaluate the model
        accuracy = accuracy_score(y_test, y_pred)
        report = classification_report(y_test, y_pred, output_dict=True)

        results.append({
            'Model': name,
            'Accuracy': accuracy,
            'Precision': report['weighted avg']['precision'],
            'Recall': report['weighted avg']['recall'],
            'F1-Score': report['weighted avg']['f1-score']
        })

    return pd.DataFrame(results)

# Assuming your dataset is already prepared (X, y)
# SMOTE for class balancing
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,
 ↪test_size=0.3, random_state=42)

# Evaluate and compare all models
results_df = evaluate_models(models, X_train, X_test, y_train, y_test)

# Display the results as a table
print(results_df)
```

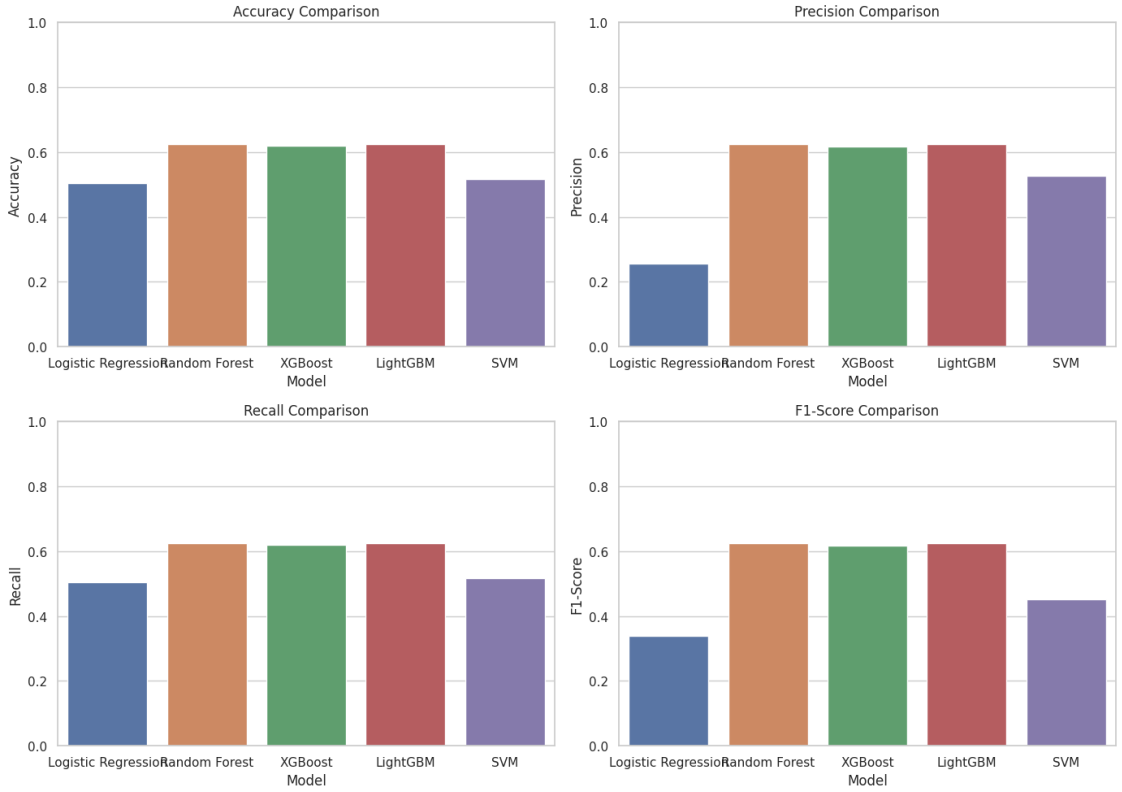
```
/opt/conda/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344:
UndefinedMetricWarning: Precision and F-score are ill-defined and being set to
0.0 in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/opt/conda/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344:
UndefinedMetricWarning: Precision and F-score are ill-defined and being set to
0.0 in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/opt/conda/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344:
UndefinedMetricWarning: Precision and F-score are ill-defined and being set to
0.0 in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

```
[LightGBM] [Info] Number of positive: 568, number of negative: 563
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of
```

```
testing was 0.000307 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3060
[LightGBM] [Info] Number of data points in the train set: 1131, number of used
features: 12
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.502210 -> initscore=0.008842
[LightGBM] [Info] Start training from score 0.008842
                 Model  Accuracy  Precision    Recall  F1-Score
0  Logistic Regression  0.505155   0.255181  0.505155  0.339076
1        Random Forest  0.624742   0.624719  0.624742  0.624720
2              XGBoost  0.618557   0.618526  0.618557  0.618518
3             LightGBM  0.624742   0.624840  0.624742  0.624749
4                  SVM  0.517526   0.526326  0.517526  0.452820
```

[17]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming `results_df` is the DataFrame from the comparison
# Here is a sample of how to visualize it

# Set plot style
sns.set(style="whitegrid")

# Create a figure and a set of subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Accuracy Plot
sns.barplot(x='Model', y='Accuracy', data=results_df, ax=axes[0, 0])
axes[0, 0].set_title('Accuracy Comparison')
axes[0, 0].set_ylim(0, 1)  # Ensuring the y-axis goes from 0 to 1

# Precision Plot
sns.barplot(x='Model', y='Precision', data=results_df, ax=axes[0, 1])
axes[0, 1].set_title('Precision Comparison')
axes[0, 1].set_ylim(0, 1)

# Recall Plot
sns.barplot(x='Model', y='Recall', data=results_df, ax=axes[1, 0])
axes[1, 0].set_title('Recall Comparison')
axes[1, 0].set_ylim(0, 1)

# F1-Score Plot
sns.barplot(x='Model', y='F1-Score', data=results_df, ax=axes[1, 1])
axes[1, 1].set_title('F1-Score Comparison')
axes[1, 1].set_ylim(0, 1)

# Adjust layout to prevent overlap
```

```
plt.tight_layout()

# Show the plot
plt.show()
```



[19]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from imblearn.over_sampling import SMOTE

# Load your dataset
data = pd.read_csv('/kaggle/input/yahoo-data/yahoo.csv')

# Create a target column: 1 if the Close price increased compared to the␣
 ↪previous day, 0 otherwise
data['price_change'] = np.where(data['Close'].diff() > 0, 1, 0)
```

```
# Feature Engineering
data['pct_change'] = (data['Close'] - data['Open']) / data['Open'] * 100
data['rolling_avg_5'] = data['Close'].rolling(window=5).mean()
data['volatility'] = data['High'] - data['Low']
window_length = 14
close_delta = data['Close'].diff()
gain = (close_delta.where(close_delta > 0, 0)).rolling(window=window_length).
 ↪mean()
loss = (-close_delta.where(close_delta < 0, 0)).rolling(window=window_length).
 ↪mean()
rs = gain / loss
data['RSI'] = 100 - (100 / (1 + rs))
ema_12 = data['Close'].ewm(span=12, adjust=False).mean()
ema_26 = data['Close'].ewm(span=26, adjust=False).mean()
data['MACD'] = ema_12 - ema_26

# Drop the first few rows with NaN values
data.dropna(inplace=True)

# Define features (X) and target (y)
X = data[['Open', 'High', 'Low', 'Close', 'Volume', 'pct_change',␣
 ↪'rolling_avg_5', 'volatility', 'RSI', 'MACD']]
y = data['price_change']

# SMOTE for class balancing
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,␣
 ↪test_size=0.3, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define the MLP model
model = Sequential()

# Add layers to the model
model.add(Dense(64, input_dim=X_train_scaled.shape[1], activation='relu'))  #␣
 ↪Input layer + first hidden layer
model.add(Dropout(0.3))  # Dropout to avoid overfitting
model.add(Dense(32, activation='relu'))  # Second hidden layer
model.add(Dropout(0.3))  # Dropout
```

```
model.add(Dense(1, activation='sigmoid'))  # Output layer (binary␣
 ↪classification)

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam',␣
 ↪metrics=['accuracy'])

# Train the model
model.fit(X_train_scaled, y_train, epochs=50, batch_size=32,␣
 ↪validation_data=(X_test_scaled, y_test), verbose=1)

# Make predictions and evaluate
y_pred = (model.predict(X_test_scaled) > 0.5).astype("int32")

# Print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
/opt/conda/lib/python3.10/site-packages/keras/src/layers/core/dense.py:87:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/50
36/36             2s 9ms/step -
accuracy: 0.5379 - loss: 0.7077 - val_accuracy: 0.5741 - val_loss: 0.6685
Epoch 2/50
36/36             0s 3ms/step -
accuracy: 0.5606 - loss: 0.6838 - val_accuracy: 0.5741 - val_loss: 0.6641
Epoch 3/50
36/36             0s 3ms/step -
accuracy: 0.5898 - loss: 0.6799 - val_accuracy: 0.5885 - val_loss: 0.6614
Epoch 4/50
36/36             0s 3ms/step -
accuracy: 0.5872 - loss: 0.6780 - val_accuracy: 0.6029 - val_loss: 0.6567
Epoch 5/50
36/36             0s 3ms/step -
accuracy: 0.5953 - loss: 0.6706 - val_accuracy: 0.5967 - val_loss: 0.6550
Epoch 6/50
36/36             0s 3ms/step -
accuracy: 0.5847 - loss: 0.6710 - val_accuracy: 0.6070 - val_loss: 0.6526
Epoch 7/50
36/36             0s 3ms/step -
accuracy: 0.5792 - loss: 0.6750 - val_accuracy: 0.6111 - val_loss: 0.6513
Epoch 8/50
36/36             0s 3ms/step -
accuracy: 0.5872 - loss: 0.6742 - val_accuracy: 0.6049 - val_loss: 0.6510
```

```
Epoch 9/50
36/36             0s 3ms/step -
accuracy: 0.6424 - loss: 0.6481 - val_accuracy: 0.6049 - val_loss: 0.6487
Epoch 10/50
36/36             0s 3ms/step -
accuracy: 0.5993 - loss: 0.6621 - val_accuracy: 0.6008 - val_loss: 0.6464
Epoch 11/50
36/36             0s 3ms/step -
accuracy: 0.6030 - loss: 0.6656 - val_accuracy: 0.6111 - val_loss: 0.6453
Epoch 12/50
36/36             0s 3ms/step -
accuracy: 0.5906 - loss: 0.6616 - val_accuracy: 0.6132 - val_loss: 0.6431
Epoch 13/50
36/36             0s 3ms/step -
accuracy: 0.6218 - loss: 0.6605 - val_accuracy: 0.6152 - val_loss: 0.6419
Epoch 14/50
36/36             0s 3ms/step -
accuracy: 0.6358 - loss: 0.6394 - val_accuracy: 0.6091 - val_loss: 0.6423
Epoch 15/50
36/36             0s 3ms/step -
accuracy: 0.6150 - loss: 0.6446 - val_accuracy: 0.6132 - val_loss: 0.6434
Epoch 16/50
36/36             0s 3ms/step -
accuracy: 0.6095 - loss: 0.6498 - val_accuracy: 0.6152 - val_loss: 0.6434
Epoch 17/50
36/36             0s 3ms/step -
accuracy: 0.6350 - loss: 0.6576 - val_accuracy: 0.6152 - val_loss: 0.6417
Epoch 18/50
36/36             0s 3ms/step -
accuracy: 0.6473 - loss: 0.6358 - val_accuracy: 0.6173 - val_loss: 0.6401
Epoch 19/50
36/36             0s 3ms/step -
accuracy: 0.5997 - loss: 0.6633 - val_accuracy: 0.6111 - val_loss: 0.6420
Epoch 20/50
36/36             0s 3ms/step -
accuracy: 0.6430 - loss: 0.6375 - val_accuracy: 0.6193 - val_loss: 0.6399
Epoch 21/50
36/36             0s 3ms/step -
accuracy: 0.6386 - loss: 0.6383 - val_accuracy: 0.6132 - val_loss: 0.6403
Epoch 22/50
36/36             0s 3ms/step -
accuracy: 0.6372 - loss: 0.6491 - val_accuracy: 0.6091 - val_loss: 0.6398
Epoch 23/50
36/36             0s 3ms/step -
accuracy: 0.6363 - loss: 0.6401 - val_accuracy: 0.6132 - val_loss: 0.6386
Epoch 24/50
36/36             0s 3ms/step -
accuracy: 0.6436 - loss: 0.6281 - val_accuracy: 0.6276 - val_loss: 0.6394
```

```
Epoch 25/50
36/36             0s 3ms/step -
accuracy: 0.6323 - loss: 0.6386 - val_accuracy: 0.6296 - val_loss: 0.6386
Epoch 26/50
36/36             0s 3ms/step -
accuracy: 0.6625 - loss: 0.6299 - val_accuracy: 0.6337 - val_loss: 0.6381
Epoch 27/50
36/36             0s 3ms/step -
accuracy: 0.6515 - loss: 0.6285 - val_accuracy: 0.6193 - val_loss: 0.6388
Epoch 28/50
36/36             0s 4ms/step -
accuracy: 0.6611 - loss: 0.6284 - val_accuracy: 0.6276 - val_loss: 0.6371
Epoch 29/50
36/36             0s 3ms/step -
accuracy: 0.6497 - loss: 0.6277 - val_accuracy: 0.6276 - val_loss: 0.6370
Epoch 30/50
36/36             0s 3ms/step -
accuracy: 0.6375 - loss: 0.6507 - val_accuracy: 0.6173 - val_loss: 0.6383
Epoch 31/50
36/36             0s 3ms/step -
accuracy: 0.6860 - loss: 0.6044 - val_accuracy: 0.6317 - val_loss: 0.6383
Epoch 32/50
36/36             0s 4ms/step -
accuracy: 0.6264 - loss: 0.6502 - val_accuracy: 0.6317 - val_loss: 0.6385
Epoch 33/50
36/36             0s 3ms/step -
accuracy: 0.6526 - loss: 0.6188 - val_accuracy: 0.6317 - val_loss: 0.6370
Epoch 34/50
36/36             0s 4ms/step -
accuracy: 0.6653 - loss: 0.6113 - val_accuracy: 0.6399 - val_loss: 0.6366
Epoch 35/50
36/36             0s 5ms/step -
accuracy: 0.6544 - loss: 0.6299 - val_accuracy: 0.6358 - val_loss: 0.6360
Epoch 36/50
36/36             0s 5ms/step -
accuracy: 0.6544 - loss: 0.6248 - val_accuracy: 0.6420 - val_loss: 0.6353
Epoch 37/50
36/36             0s 5ms/step -
accuracy: 0.6369 - loss: 0.6442 - val_accuracy: 0.6420 - val_loss: 0.6360
Epoch 38/50
36/36             0s 3ms/step -
accuracy: 0.6457 - loss: 0.6411 - val_accuracy: 0.6440 - val_loss: 0.6361
Epoch 39/50
36/36             0s 3ms/step -
accuracy: 0.6465 - loss: 0.6221 - val_accuracy: 0.6461 - val_loss: 0.6354
Epoch 40/50
36/36             0s 3ms/step -
accuracy: 0.6854 - loss: 0.6105 - val_accuracy: 0.6461 - val_loss: 0.6356
```

```
Epoch 41/50
36/36             0s 4ms/step -
accuracy: 0.6339 - loss: 0.6317 - val_accuracy: 0.6317 - val_loss: 0.6366
Epoch 42/50
36/36             0s 3ms/step -
accuracy: 0.6534 - loss: 0.6169 - val_accuracy: 0.6379 - val_loss: 0.6372
Epoch 43/50
36/36             0s 3ms/step -
accuracy: 0.6554 - loss: 0.6284 - val_accuracy: 0.6379 - val_loss: 0.6345
Epoch 44/50
36/36             0s 3ms/step -
accuracy: 0.6626 - loss: 0.6166 - val_accuracy: 0.6379 - val_loss: 0.6356
Epoch 45/50
36/36             0s 3ms/step -
accuracy: 0.6501 - loss: 0.6155 - val_accuracy: 0.6420 - val_loss: 0.6368
Epoch 46/50
36/36             0s 3ms/step -
accuracy: 0.6255 - loss: 0.6470 - val_accuracy: 0.6399 - val_loss: 0.6382
Epoch 47/50
36/36             0s 3ms/step -
accuracy: 0.6232 - loss: 0.6370 - val_accuracy: 0.6379 - val_loss: 0.6338
Epoch 48/50
36/36             0s 3ms/step -
accuracy: 0.6702 - loss: 0.6127 - val_accuracy: 0.6358 - val_loss: 0.6340
Epoch 49/50
36/36             0s 3ms/step -
accuracy: 0.6701 - loss: 0.6130 - val_accuracy: 0.6399 - val_loss: 0.6329
Epoch 50/50
36/36             0s 4ms/step -
accuracy: 0.6602 - loss: 0.6185 - val_accuracy: 0.6358 - val_loss: 0.6348
16/16             0s 4ms/step

Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.60      0.63       248
           1       0.62      0.67      0.64       238

    accuracy                           0.64       486
   macro avg       0.64      0.64      0.64       486
weighted avg       0.64      0.64      0.64       486
```

[20]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from imblearn.over_sampling import SMOTE

# Load your dataset
data = pd.read_csv('/kaggle/input/yahoo-data/yahoo.csv')

# Create a target column: 1 if the Close price increased compared to the␣
 ↪previous day, 0 otherwise
data['price_change'] = np.where(data['Close'].diff() > 0, 1, 0)

# Feature Engineering
data['pct_change'] = (data['Close'] - data['Open']) / data['Open'] * 100
data['rolling_avg_5'] = data['Close'].rolling(window=5).mean()
data['volatility'] = data['High'] - data['Low']
window_length = 14
close_delta = data['Close'].diff()
gain = (close_delta.where(close_delta > 0, 0)).rolling(window=window_length).
 ↪mean()
loss = (-close_delta.where(close_delta < 0, 0)).rolling(window=window_length).
 ↪mean()
rs = gain / loss
data['RSI'] = 100 - (100 / (1 + rs))
ema_12 = data['Close'].ewm(span=12, adjust=False).mean()
ema_26 = data['Close'].ewm(span=26, adjust=False).mean()
data['MACD'] = ema_12 - ema_26

# Drop the first few rows with NaN values due to rolling calculations
data.dropna(inplace=True)

# Define features (X) and target (y)
X = data[['Open', 'High', 'Low', 'Close', 'Volume', 'pct_change',␣
 ↪'rolling_avg_5', 'volatility', 'RSI', 'MACD']]
y = data['price_change']

# SMOTE for class balancing
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Reshape the data for LSTM
# We need to create sequences of data (e.g., last 10 timesteps)
def create_sequences(X, y, time_steps=10):
    Xs, ys = [], []
    for i in range(len(X) - time_steps):
        Xs.append(X[i:i + time_steps])
        ys.append(y[i + time_steps])
```

```
    return np.array(Xs), np.array(ys)

time_steps = 10  # Number of timesteps (e.g., look back at last 10 days)
X_seq, y_seq = create_sequences(X_resampled, y_resampled, time_steps)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_seq, y_seq, test_size=0.
 ↪3, random_state=42)

# Standardize the features (LSTM performs better with scaled data)
scaler = StandardScaler()
X_train = X_train.reshape(-1, X_train.shape[2])  # Flatten to 2D
X_train = scaler.fit_transform(X_train)  # Scale
X_train = X_train.reshape(-1, time_steps, X_seq.shape[2])  # Reshape back to 3D

X_test = X_test.reshape(-1, X_test.shape[2])  # Flatten to 2D
X_test = scaler.transform(X_test)  # Scale
X_test = X_test.reshape(-1, time_steps, X_seq.shape[2])  # Reshape back to 3D

# Define the LSTM model
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(time_steps, X_seq.
 ↪shape[2])))  # First LSTM layer
model.add(Dropout(0.3))
model.add(LSTM(50, return_sequences=False))  # Second LSTM layer
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))  # Output layer (binary␣
 ↪classification)

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',␣
 ↪metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=20, batch_size=32,␣
 ↪validation_data=(X_test, y_test), verbose=1)

# Make predictions
y_pred = (model.predict(X_test) > 0.5).astype("int32")

# Print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Epoch 1/20

/opt/conda/lib/python3.10/site-packages/keras/src/layers/rnn/rnn.py:204:
```

```
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)
36/36            5s 28ms/step -
accuracy: 0.4836 - loss: 0.6941 - val_accuracy: 0.5238 - val_loss: 0.6922
Epoch 2/20
36/36            1s 13ms/step -
accuracy: 0.6034 - loss: 0.6721 - val_accuracy: 0.5466 - val_loss: 0.6951
Epoch 3/20
36/36            1s 14ms/step -
accuracy: 0.6024 - loss: 0.6593 - val_accuracy: 0.5901 - val_loss: 0.6791
Epoch 4/20
36/36            1s 15ms/step -
accuracy: 0.6138 - loss: 0.6627 - val_accuracy: 0.6046 - val_loss: 0.6705
Epoch 5/20
36/36            1s 14ms/step -
accuracy: 0.6388 - loss: 0.6487 - val_accuracy: 0.6377 - val_loss: 0.6479
Epoch 6/20
36/36            1s 13ms/step -
accuracy: 0.6827 - loss: 0.6132 - val_accuracy: 0.6667 - val_loss: 0.6231
Epoch 7/20
36/36            1s 13ms/step -
accuracy: 0.6888 - loss: 0.5858 - val_accuracy: 0.7039 - val_loss: 0.5680
Epoch 8/20
36/36            1s 13ms/step -
accuracy: 0.7348 - loss: 0.5355 - val_accuracy: 0.7557 - val_loss: 0.4906
Epoch 9/20
36/36            1s 13ms/step -
accuracy: 0.7796 - loss: 0.4525 - val_accuracy: 0.8261 - val_loss: 0.4021
Epoch 10/20
36/36            1s 13ms/step -
accuracy: 0.8267 - loss: 0.3782 - val_accuracy: 0.8986 - val_loss: 0.3155
Epoch 11/20
36/36            1s 13ms/step -
accuracy: 0.8858 - loss: 0.2761 - val_accuracy: 0.8923 - val_loss: 0.2508
Epoch 12/20
36/36            1s 14ms/step -
accuracy: 0.8763 - loss: 0.2553 - val_accuracy: 0.9172 - val_loss: 0.2089
Epoch 13/20
36/36            1s 14ms/step -
accuracy: 0.9299 - loss: 0.2016 - val_accuracy: 0.9234 - val_loss: 0.1852
Epoch 14/20
36/36            1s 13ms/step -
accuracy: 0.9317 - loss: 0.1628 - val_accuracy: 0.9337 - val_loss: 0.1677
Epoch 15/20
36/36            1s 13ms/step -
```

```
accuracy: 0.9439 - loss: 0.1483 - val_accuracy: 0.9379 - val_loss: 0.1589
Epoch 16/20
36/36             1s 13ms/step -
accuracy: 0.9500 - loss: 0.1394 - val_accuracy: 0.9358 - val_loss: 0.1636
Epoch 17/20
36/36             1s 15ms/step -
accuracy: 0.9397 - loss: 0.1563 - val_accuracy: 0.9317 - val_loss: 0.1650
Epoch 18/20
36/36             1s 14ms/step -
accuracy: 0.9397 - loss: 0.1665 - val_accuracy: 0.9358 - val_loss: 0.1527
Epoch 19/20
36/36             1s 14ms/step -
accuracy: 0.9453 - loss: 0.1268 - val_accuracy: 0.9420 - val_loss: 0.1507
Epoch 20/20
36/36             1s 13ms/step -
accuracy: 0.9525 - loss: 0.1329 - val_accuracy: 0.9400 - val_loss: 0.1441
16/16             1s 24ms/step

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.94      0.94       252
           1       0.94      0.94      0.94       231

    accuracy                           0.94       483
   macro avg       0.94      0.94      0.94       483
weighted avg       0.94      0.94      0.94       483
```

[21]:
```python
import pandas as pd

# Assuming you have the classification reports or metrics for each model
# Update these values with the actual performance metrics for each model

model_performance = {
    'Model': ['Logistic Regression', 'Random Forest', 'XGBoost', 'LightGBM',␣
 ↪'SVM', 'LSTM'],
    'Accuracy': [0.505, 0.625, 0.619, 0.625, 0.518, 0.94],  # Accuracy for each␣
 ↪model
    'Precision': [0.255, 0.625, 0.619, 0.625, 0.526, 0.94],  # Weighted avg␣
 ↪precision for each model
    'Recall': [0.505, 0.625, 0.619, 0.625, 0.518, 0.94],  # Weighted avg recall␣
 ↪for each model
    'F1-Score': [0.339, 0.625, 0.619, 0.625, 0.453, 0.94]  # Weighted avg␣
 ↪F1-score for each model
}
```

```
# Convert to DataFrame
results_df = pd.DataFrame(model_performance)

# Display the table
print(results_df)
```

```
                Model  Accuracy  Precision  Recall  F1-Score
0  Logistic Regression     0.505      0.255   0.505     0.339
1        Random Forest     0.625      0.625   0.625     0.625
2              XGBoost     0.619      0.619   0.619     0.619
3             LightGBM     0.625      0.625   0.625     0.625
4                  SVM     0.518      0.526   0.518     0.453
5                 LSTM     0.940      0.940   0.940     0.940
```

[22]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming you have the classification reports or metrics for each model
# Replace these values with the actual performance metrics from your models
model_performance = {
    'Model': ['Logistic Regression', 'Random Forest', 'XGBoost', 'LightGBM',␣
 ↪'SVM', 'LSTM'],
    'Accuracy': [0.505, 0.625, 0.619, 0.625, 0.518, 0.94],  # Accuracy for each␣
 ↪model
    'Precision': [0.255, 0.625, 0.619, 0.625, 0.526, 0.94],  # Weighted avg␣
 ↪precision for each model
    'Recall': [0.505, 0.625, 0.619, 0.625, 0.518, 0.94],  # Weighted avg recall␣
 ↪for each model
    'F1-Score': [0.339, 0.625, 0.619, 0.625, 0.453, 0.94]  # Weighted avg␣
 ↪F1-score for each model
}

# Convert to DataFrame
results_df = pd.DataFrame(model_performance)

# Set plot style
sns.set(style="whitegrid")

# Create a figure and a set of subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Accuracy Plot
sns.barplot(x='Model', y='Accuracy', data=results_df, ax=axes[0, 0])
axes[0, 0].set_title('Accuracy Comparison')
axes[0, 0].set_ylim(0, 1)  # Ensuring the y-axis goes from 0 to 1
```

```
# Precision Plot
sns.barplot(x='Model', y='Precision', data=results_df, ax=axes[0, 1])
axes[0, 1].set_title('Precision Comparison')
axes[0, 1].set_ylim(0, 1)

# Recall Plot
sns.barplot(x='Model', y='Recall', data=results_df, ax=axes[1, 0])
axes[1, 0].set_title('Recall Comparison')
axes[1, 0].set_ylim(0, 1)

# F1-Score Plot
sns.barplot(x='Model', y='F1-Score', data=results_df, ax=axes[1, 1])
axes[1, 1].set_title('F1-Score Comparison')
axes[1, 1].set_ylim(0, 1)

# Adjust layout to prevent overlap
plt.tight_layout()

# Show the plot
plt.show()
```